

Sau bài thực hành này, sinh viên có thể

- Cài đặt và cấu hình Apache Spark
- Viết được chương trình tạo Spark Session
- Viết được chương trình xử lý RDD của Spark

1. CÀI ĐẶT & CẤU HÌNH APACHE SPARK

1.1. Cài đặt PyCharm Community

<https://www.jetbrains.com/pycharm/download/download-thanks.html?platform=windows&code=PCC>

1.2. Cài đặt Java 8

<https://www.oracle.com/technetwork/java/javase/downloads/jdk8-downloads-2133151.html>

Sau khi cài đặt Java vào máy, ta tạo các biến môi trường như sau

```
JAVA_HOME = C:\Program Files\Java\jdk1.8.0_201
PATH = %PATH%;C:\Program Files\Java\jdk1.8.0_201\bin
```

1.3. Cài đặt PySpark

- Truy cập trang <https://spark.apache.org/downloads.html>
- Tải gói Apache Spark

Download Apache Spark™

- Choose a Spark release:
- Choose a package type:
- Download Spark: [spark-3.0.0-bin-hadoop2.7.tgz](#)
- Verify this release using the 3.0.0 [signatures](#), [checksums](#) and [project release KEYS](#).

- Tạo thư mục C:\apps và giải nén tập tin spark-3.0.0-bin-hadoop2.7.tgz vừa tải vào thư mục
- Thiết lập biến môi trường như sau

```
SPARK_HOME = C:\apps\spark-3.0.0-bin-hadoop2.7
HADOOP_HOME = C:\apps\spark-3.0.0-bin-hadoop2.7
PATH=%PATH%;C:\apps\spark-3.0.0-bin-hadoop2.7\bin
```

- Chép tập tin winutils.exe vào thư mục %SPARK_HOME%\bin

1.4. Kiểm tra cấu hình Apache Spark thành công

- Mở cửa sổ command prompt

- Gõ lệnh pyspark

```
C:\Users\Admin>pyspark
Python 3.9.12 (tags/v3.9.12:b28265d, Mar 23 2022, 23:52:46) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
Welcome to

      /_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_\
     /  /  /  /  /  /  /  /  /  /  /
    /    /    /    /    /    /    /
   /      /      /      /      /      /
  /        /        /        /        /
 /          /          /          /          /
/            /            /            /            /
 \          /            /            /            /
  \        /          /          /          /          /
   \      /        /        /        /        /
    \    /      /      /      /      /      /
     \  /    /    /    /    /    /    /    /
      \_/_/_/_/_/_/_/_/_/_/_/_/_/_/_/_\

version 3.5.0

Using Python version 3.9.12 (tags/v3.9.12:b28265d, Mar 23 2022 23:52:46)
Spark context Web UI available at http://DESKTOP-U523NNL:4040
Spark context available as 'sc' (master = local[*], app id = local-1703411987854).
SparkSession available as 'spark'.
>>>
```

1.5. Cài đặt gói pyspark hỗ trợ lập trình

```
pip install pyspark
```

2. VIẾT CHƯƠNG TRÌNH XỬ LÝ RDD CƠ BẢN

RDD (Resilient Distributed Dataset) là tập hợp các đối tượng thành danh sách. Khi xử lý, các đối tượng sẽ được phân thành nhiều mảnh tới các máy tính (node) trong mạng (gọi là cluster). Các node này xử lý dữ liệu song song.

2.1. Tao dãy số nguyên và in ra màn hình

```
from pyspark.sql import SparkSession

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [1,2,3,4,5,6,7,8,9,10,11,12]
rdd=spark.sparkContext.parallelize(data)
print(rdd.collect())
```

2.2. Xem số partition đang hiện có

```
from pyspark.sql import SparkSession

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [1,2,3,4,5,6,7,8,9,10,11,12]
rdd=spark.sparkContext.parallelize(data)
print("The number of partitions:" + str(rdd.getNumPartitions()))
print(rdd.collect())
```

2.3. Thay đổi số phân hoạch

```
from pyspark.sql import SparkSession
import findspark
findspark.init()

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [1,2,3,4,5,6,7,8,9,10,11,12]
rdd=spark.sparkContext.parallelize(data)
repartition = rdd.repartition(4)
print('The new number of partitions:' + str(repartition.getNumPartitions()))
print(repartition.collect())
```

2.4. Dùng hàm map() thay đổi nội dung RDD

```

from pyspark.sql import SparkSession
import findspark
findspark.init()

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [1,2,3,4,5,6,7,8,9,10,11,12]
rdd=spark.sparkContext.parallelize(data)
rdd2 = rdd.map(lambda x: x+1)
print(rdd2.collect())

```

2.5. Dùng hàm flatMap() làm phẳng nội dung của RDD

```

from pyspark.sql import SparkSession
import findspark
findspark.init()

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [
    [1, 2, 3, 4, 5],
    [9, 8, 7, 6, 10]
]

rdd=spark.sparkContext.parallelize(data)
rdd2 = rdd.flatMap(lambda x: x * 2)
print(rdd2.collect())

```

2.6. Lọc phần tử trong RDD

```

from pyspark.sql import SparkSession
import findspark
findspark.init()

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [1,2,3,4,5,6,7,8,9,10,11,12]
rdd = spark.sparkContext.parallelize(data)
rdd2 = rdd.filter(lambda x: x % 2 == 0)
print(rdd2.collect())

```

2.7. Ánh xạ hàm vào phân hoạch

```

from pyspark.sql import SparkSession
import findspark
findspark.init()

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [1,2,3,4,5,6,7,8,9,10,11,12]
rdd = spark.sparkContext.parallelize(data, 2)
def checkEvenNumber(partition):
    temp = []
    for i in partition:
        if i % 2 == 0:
            temp.append(i * 2)
    return iter(temp)
rdd2 = rdd.mapPartitions(checkEvenNumber)
print(rdd2.collect())

```

2.8. Ánh xạ hàm vào phân hoạch có chỉ số

```

from pyspark.sql import SparkSession
import findspark
findspark.init()

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [1,2,3,4,5,6,7,8,9,10,11,12]
rdd = spark.sparkContext.parallelize(data, 2)
def checkEvenNumber(partition_index, iter):
    temp = []
    for i in iter:
        if i % 2 == 0:
            temp.append((partition_index, i, '--->', i * 3))
    return temp
rdd2 = rdd.mapPartitionsWithIndex(checkEvenNumber)
print(rdd2.collect())

```

2.9. Nối hai RDD

```

from pyspark.sql import SparkSession
import findspark
findspark.init()

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [1,2,3,4,5,6,7,8,9,10,11,12]
data1 = [1,22,34,45,5,6,72,8,90,100,101,124]
rdd = spark.sparkContext.parallelize(data)
rdd1 = spark.sparkContext.parallelize(data1)
rdd2 = rdd.union(rdd1)
print(rdd2.collect())

```

2.10. Giao hai RDD

```

from pyspark.sql import SparkSession
import findspark
findspark.init()

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [1,2,3,4,5,6,7,8,9,10,11,12]
data1 = [1,22,34,45,5,6,72,8,90,100,101,124]
rdd = spark.sparkContext.parallelize(data)
rdd1 = spark.sparkContext.parallelize(data1)
rdd2 = rdd.intersection(rdd1)
print(rdd2.collect())

```

2.11. Hiệu 2 RDD

```

from pyspark.sql import SparkSession
import findspark
findspark.init()

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [1,2,3,4,5,6,7,8,9,10,11,12]
data1 = [1,22,34,45,5,6,72,8,90,100,101,124]
rdd = spark.sparkContext.parallelize(data)
rdd1 = spark.sparkContext.parallelize(data1)
rdd2 = rdd.subtract(rdd1)
print(rdd2.collect())

```

2.12. Lấy mẫu từ RDD

```
from pyspark.sql import SparkSession
import findspark
findspark.init()

spark = SparkSession.builder\
    .master('local[1]')\
    .appName('bai_thuc_hanh_1.com')\
    .getOrCreate()

data = [1,2,3,4,5,6,7,8,9,10,11,12]
data1 = [1,22,34,45,5,6,72,8,90,100,101,124]
rdd = spark.sparkContext.parallelize(data)
rdd1 = spark.sparkContext.parallelize(data1)
rdd2 = rdd.sample(True, 0.3)
print(rdd2.collect())
```

3. BÀI TẬP

3.1. Viết chương trình tạo dữ liệu gồm 1.000.000 số nguyên ngẫu nhiên trong khoảng $[0, 1.000]$. Cho biết có bao nhiêu số chẵn, bao nhiêu số lẻ. Hãy viết ứng dụng trên Tkinter.

3.2. Viết chương trình tạo dữ liệu gồm 1.000.000 số nguyên ngẫu nhiên trong khoảng $[-1.000, 1.000]$. Thực hiện lấy số chẵn $\times 3$, số lẻ $\times 2$, số âm lẻ $\times 4$, số âm chẵn $\times 5$. So với dữ liệu ban đầu, có bao nhiêu số chia hết cho 2, 3, 5. Viết ứng dụng trên Tkinter.

3.3. Số bảo hiểm xã hội của Canada (SIN - Canadian Social Insurance Number) là một số có 9 chữ số, được kiểm tra tính hợp lệ như sau:

- Số phải nhất (vị trí 1, tính từ phải sang), là số kiểm tra (check digit).
- Trọng số được tính từ phải qua trái (không tính check digit), bằng $s1 + s2$:

+ $s1$ là tổng các số có vị trí lẻ.

+ Các số có vị trí chẵn nhân đôi. Nếu kết quả nhân đôi có hai chữ số thì kết quả là tổng của hai chữ số này. $s2$ là tổng các kết quả.

SIN hợp lệ có tổng trọng số với số kiểm tra chia hết cho 10.

Ví dụ: SIN 193456787

- Số kiểm tra là 7 (số tô đậm).

- Trọng số là tổng của $s1$ và $s2$, với:

$$+ s1 = 1 + 3 + 5 + 7 = 16$$

+ Các số có vị trí chẵn nhân đôi:

$$(9 \times 2) (4 \times 2) (6 \times 2) (8 \times 2) \rightarrow 18 \ 8 \ 12 \ 16$$

$$s2 = (1 + 8) + 8 + (1 + 2) + (1 + 6) = 27$$

$$\text{Trọng số bằng } s1 + s2 = 16 + 27 = 43.$$

Vì tổng trọng số với số kiểm tra $43 + 7 = 50$ chia hết cho 10 nên số SIN hợp lệ

Viết chương trình kiểm tra số SIN hợp lệ trên Tkinter

- 3.4. Viết chương trình tạo dữ liệu gồm 1.000.000 số nguyên ngẫu nhiên trong khoảng $[0, 1.000]$. Cho biết các ước và bội (có xuất hiện trong dữ liệu) của mỗi phần tử. Hãy viết ứng dụng trên Tkinter.
- 3.5. Viết chương trình tạo dữ liệu gồm 1.000.000 số nguyên ngẫu nhiên trong khoảng $[0, 1.000]$. Tách thành 3 tập dữ liệu (30%, 30%, 40%). Liệt kê những phần tử chung của 3 tập. Viết ứng dụng trên Tkinter.
- 3.6. Viết chương trình tạo dữ liệu gồm 1.000.000 số nguyên ngẫu nhiên trong khoảng $[0, 1.000]$. Có bao nhiêu số có thể phân tích được thành thừa số nguyên tố (các số nguyên tố có xuất hiện trong dữ liệu). Viết ứng dụng trên Tkinter.
- 3.7. Viết chương trình tạo dữ liệu gồm 1.000.000 số nguyên ngẫu nhiên trong khoảng $[0, 1.000]$ với 4 phân hoạch. Có bao nhiêu số 1000 trong mỗi phân hoạch. Viết ứng dụng trên Tkinter.
- 3.8. Viết chương trình tạo dữ liệu gồm 1.000.000 số nguyên ngẫu nhiên trong khoảng $[0, 1.000]$ với 3 phân hoạch. Phân hoạch 1 gồm những số chia hết cho 5. Phân hoạch 2 gồm những số là bội của phân hoạch 1, phân hoạch 3 gồm những số là ước của phân hoạch 1. Viết ứng dụng trên Tkinter.